```
import com.polarbookshop.orderservice.config.DataConfig;
import org.junit.jupiter.api.Test;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;
import reactor.test.StepVerifier;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.r2dbc.DataR2dbcTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;

@DataR2dbcTest                                                    ❶
@Import(DataConfig.class)                                         ❷
@Testcontainers                                                   ❸
class OrderRepositoryR2dbcTests {

  @Container                                                      ❹
  static PostgreSQLContainer<?> postgresql =
    new PostgreSQLContainer<>(DockerImageName.parse("postgres:14.4"));

  @Autowired
  private OrderRepository orderRepository;

  @DynamicPropertySource                                          ❺
  static void postgresqlProperties(DynamicPropertyRegistry registry) {
    registry.add("spring.r2dbc.url", OrderRepositoryR2dbcTests::r2dbcUrl);
    registry.add("spring.r2dbc.username", postgresql::getUsername);
    registry.add("spring.r2dbc.password", postgresql::getPassword);
    registry.add("spring.flyway.url", postgresql::getJdbcUrl);
  }

  private static String r2dbcUrl() {                              ❻
    return String.format("r2dbc:postgresql://%s:%s/%s",
      postgresql.getContainerIpAddress(),
      postgresql.getMappedPort(PostgreSQLContainer.POSTGRESQL_PORT),
      postgresql.getDatabaseName());
  }

  @Test
  void createRejectedOrder() {
    var rejectedOrder = OrderService.buildRejectedOrder("1234567890", 3);
    StepVerifier
      .create(orderRepository.save(rejectedOrder))                ❼
      .expectNextMatches(                                         ❽
        order -> order.status().equals(OrderStatus.REJECTED))
      .verifyComplete();                                          ❾
  }
}
```